

ISEN

ALL IS DIGITAL!

BREST

DP CYBERSÉCURITÉ 2025 - 2026

Détection de menaces dans le trafic réseau avec **TabTransformer**

RÉALISÉ PAR :

Agathe Dormoy - Jérémie HSU - Naoufel Benidar - Vincent Dutournier

Table des matières

1	Choix des features	4
1.1	Heatmap	4
1.2	Analyse de la séparation multi-classe (ANOVA)	5
1.3	Importance des features	5
1.4	Redondance entre features	6
1.5	Choix final	7
2	Nettoyage des données	8
2.1	L'ouverture des fichiers	8
2.2	Les valeurs aberrantes et différence d'écriture	9
2.3	Nettoyage des données	9
3	Choix des métriques d'évaluation des modèles	11
3.1	Accuracy score	11
3.2	F1-score	11
3.3	Matrice de confusion	11
3.4	Courbe ROC et AUC	12
4	Modèle : Random Forest Classifieur avec Équilibrage de Données	13
4.1	Méthodologie	13
4.2	Résultats et Analyse	14
5	Modèle : XGBoost	15
5.1	Préparation des Données	15
5.2	Architecture du Modèle et Choix des Hyperparamètres	15
5.3	Suivi de la Convergence	15
5.4	Résultats et Analyse	16
6	Modèle : TabNet	17
6.1	Préparation des Données	17
6.2	Architecture et Mécanisme d'Attention	17

6.3	Stratégie d'Optimisation	18
6.4	Analyse des Résultats	18
7	Modèle : TabTransformer	20
7.1	Qu'est-ce qu'un TabTransformer?	20
7.2	Préparation des Données	20
7.3	Architecture et Mécanisme d'Attention	21
7.4	Résultats et Analyse	22
7.5	Interprétabilité : Analyse des Attention Maps	23
7.6	Tests de Robustesse	24
8	Conclusion	26

1 Choix des features

La sélection des features est une étape clé dans ce travail, car elle influence directement les performances et la stabilité des modèles de détection d'intrusions. L'objectif n'est pas de conserver toutes les variables disponibles, mais d'identifier celles qui apportent réellement de l'information tout en évitant la redondance. Cette démarche a été appliquée sur deux jeux de données complémentaires : UNSW-NB15, qui contient plusieurs catégories d'attaques (classification multi-classe via `attack_cat`), et CIC-DDoS2019, spécialisé dans les attaques DDoS.

1.1 Heatmap

Pour visualiser les relations globales entre variables, nous avons utilisé des matrices de corrélation sous forme de heatmap. Une heatmap représente graphiquement les coefficients de corrélation entre les variables : plus la couleur est intense, plus la relation linéaire est forte. Cela permet d'identifier rapidement des groupes de variables fortement liées entre elles, et donc potentiellement redondantes.

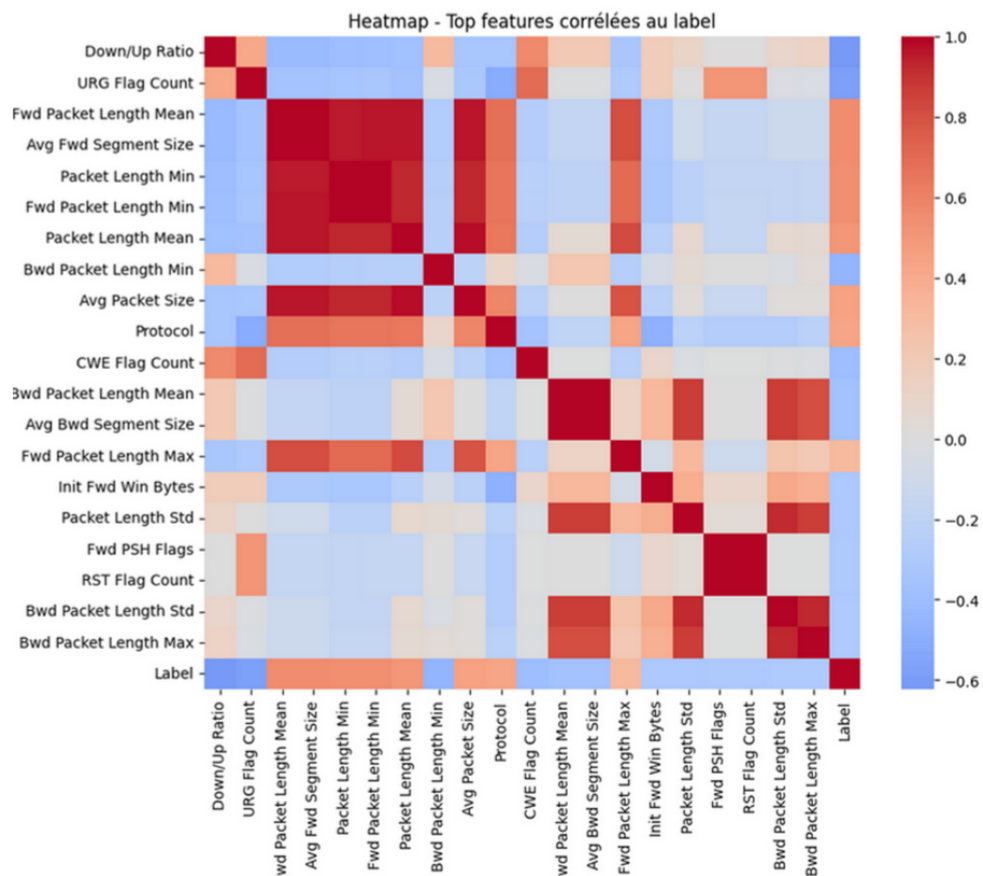


FIGURE 1 : Heatmap des features du dataset CIC-DDoS2019

Dans la première version de notre travail, nous utilisons principalement la corrélation de Pearson

avec un label binaire. Cependant, dans le cas de UNSW-NB15, la cible `attack_cat` comporte neuf classes sans ordre naturel. Utiliser Pearson aurait supposé un ordre artificiel entre ces classes, ce qui aurait biaisé l'analyse. Nous avons donc adapté notre méthodologie. Sur UNSW-NB15, on observe que de nombreuses variables contextuelles de type `ct_` sont corrélées entre elles, ce qui reflète leur proximité fonctionnelle. Sur CIC-DDoS2019, les corrélations fortes apparaissent principalement entre les métriques de volume et de débit. Cette analyse nous a surtout servi à détecter la redondance, plutôt qu'à mesurer directement le pouvoir discriminant.

1.2 Analyse de la séparation multi-classe (ANOVA)

Pour évaluer le pouvoir discriminant des variables numériques dans le cas multi-classe, nous avons utilisé le test ANOVA (F-test). Contrairement à Pearson, l'ANOVA ne suppose aucun ordre entre les classes. Elle mesure si la moyenne d'une variable diffère significativement d'une classe à l'autre. Plus le F-score est élevé, plus la variable sépare bien les différentes catégories d'attaque.

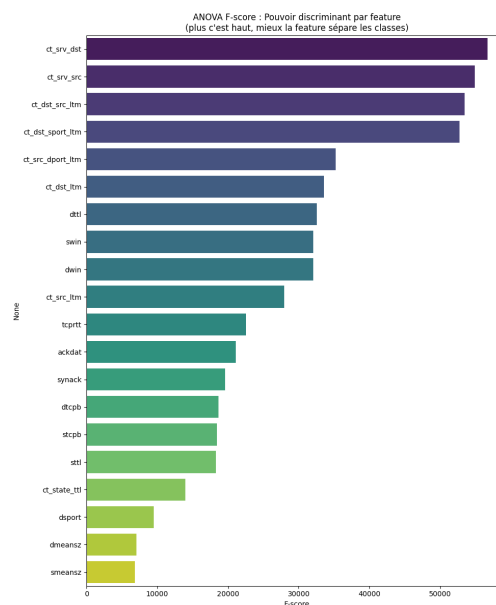


Figure 2 : Top features selon le score ANOVA (UNSW-NB15)

Les résultats montrent que les variables contextuelles comme `ct_srv_dst`, `ct_dst_src_ltm` ou `ct_srv_src` obtiennent des scores très élevés. Cela signifie que leur distribution varie fortement selon le type d'attaque, ce qui confirme leur intérêt pour une classification multi-classe. Cette approche permet donc d'identifier les variables réellement sensibles à la nature de l'attaque, et pas seulement à la distinction normal/malveillant.

1.3 Importance des features

L'ANOVA évalue chaque variable individuellement. Pour aller plus loin, nous avons utilisé deux approches complémentaires : l'information mutuelle et l'importance issue d'un Random

Forest. L'information mutuelle mesure la quantité d'information qu'une variable apporte sur la classe, sans supposer de relation linéaire. Elle permet aussi de comparer directement variables numériques et catégorielles sur la même échelle. On observe par exemple que dsport, sbytes et smeansz figurent parmi les variables les plus informatives.

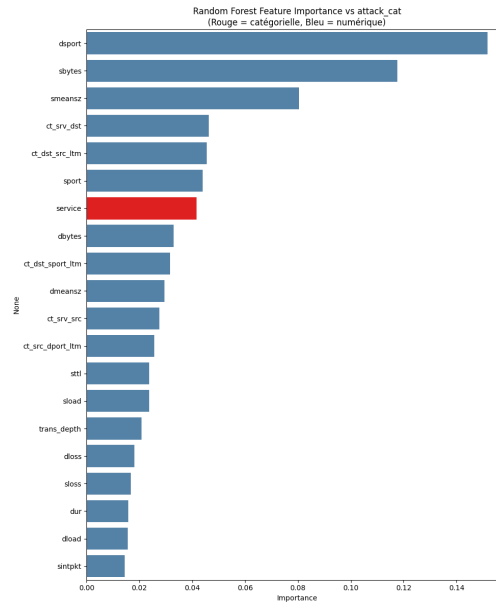


Figure 3 : Top features les plus importantes (UNSW-NB15)

Le Random Forest, entraîné avec un `class_weight='balanced'` pour compenser le fort déséquilibre (la classe Generic représentant plus de 60 % des données), permet d'évaluer l'importance des variables dans un contexte prédictif réel. Les résultats confirment le rôle central de dsport, sbytes, ainsi que des variables contextuelles ct_.

Pour les variables catégorielles, nous avons utilisé le coefficient de Cramér's V afin de mesurer directement leur association avec attack_cat sans passer par un encodage numérique. Les variables service et proto montrent une association significative, contrairement à state qui ressort plus faible dans le ranking combiné.

1.4 Redondance entre features

Une fois les variables discriminantes identifiées, nous avons étudié la redondance entre features. Certaines variables décrivent des phénomènes très proches, notamment au sein des groupes ct_ ou des métriques liées aux tailles de paquets. Lorsque deux variables présentent une corrélation très élevée, conserver les deux n'apporte pas d'information supplémentaire significative et peut augmenter la complexité du modèle.

1.5 Choix final

Au final, le choix des features repose sur une combinaison de plusieurs méthodes complémentaires : ANOVA pour mesurer la séparation multi-classe, information mutuelle pour capturer les relations non linéaires, importance Random Forest pour évaluer l'impact prédictif réel, et Cramér's V pour analyser proprement les variables catégorielles.

Plutôt que de dépendre d'une seule métrique, nous avons normalisé et combiné ces scores afin d'obtenir un ranking global plus robuste. Une variable qui ressort dans plusieurs méthodes a ainsi plus de chances d'être réellement pertinente.

Pour UNSW-NB15, 20 features numériques et 4 catégorielles ont été retenues, notamment dsport, sbytes, les variables contextuelles ct_ ainsi que certaines métriques TCP comme synack ou tcprtt. Pour CIC-DDoS2019, 20 features ont également été conservées. Cette sélection permet de conserver l'essentiel de l'information discriminante tout en limitant la complexité et le risque de surapprentissage.

Dataset	Type	Features retenues
UNSW-NB15	Numériques	dsport, sbytes, smeansz, ct_srv_dst, ct_dst_src_ltm, ct_srv_src, ct_dst_sport_ltm, ct_src_dport_ltm, ct_dst_ltm, sport, ct_src_ltm, sload, dttl, dwin, swin, tcprtt, synack, ackdat, dmeansz, dbytes
UNSW-NB15	Catégorielles	state, service, proto, attack_cat
CIC-DDoS2019	Numériques	Flow Duration, Total Fwd Packets, Total Backward Packets, Fwd Packet Length Min, Bwd Packet Length Mean, Packet Length Std, Flow Bytes/s, Flow Packets/s, Flow IAT Mean, Flow IAT Std, Down/Up Ratio, SYN Flag Count, ACK Flag Count, URG Flag Count, Init Fwd Win Bytes, Init Bwd Win Bytes, Active Mean, Idle Mean, Bwd Packets/s
CIC-DDoS2019	Catégorielles	Protocol

2 Nettoyage des données

La première étape de ce projet a consisté à nettoyer les données afin d'obtenir un dataset fiable pour la sélection des features et l'entraînement du modèle d'intelligence artificielle.

Le nettoyage vise à garantir l'intégrité, la cohérence et l'uniformité des données. Il comprend principalement la gestion des valeurs nulles, l'harmonisation des catégories écrites différemment et la suppression des valeurs aberrantes. Ces étapes sont indispensables pour éviter les biais, assurer la qualité des analyses et améliorer la performance du modèle.

Le développement a été réalisé en Python, en utilisant les bibliothèques NumPy, Pandas, os et json pour la manipulation des données, la gestion des fichiers et l'exploitation du fichier de configuration des features.

L'éditeur Visual Studio Code a été utilisé pour faciliter la gestion de l'arborescence et des

fichiers. Enfin, GitHub a servi au partage et au versionnage des datasets et du code du projet.

2.1 L'ouverture des fichiers

L'ouverture des fichiers s'effectue à l'aide de la bibliothèque `os`. La fonction `listdir()` permet de lister les fichiers présents dans un répertoire donné. Il est ensuite possible de récupérer l'extension de chaque fichier grâce à `path.splitext()`, afin de déterminer la méthode de lecture appropriée.

Selon le type détecté, les fichiers sont ouverts avec `pd.read_csv()` ou `pd.read_parquet()`. Le mot-clé `yield` est utilisé afin de traiter les fichiers un par un, en retournant à chaque itération le dataset, son nom et son chemin complet. Cette approche permet une gestion plus efficace de la mémoire et facilite le traitement de plusieurs fichiers successivement.

Le traitement des valeurs nulles a été divisé en deux étapes distinctes :

1. Vérification de la présence de valeurs nulles ;
2. Suppression des lignes concernées si nécessaire.

Avant la vérification, plusieurs représentations textuelles sont remplacées par la valeur standard `NA` afin d'uniformiser leur détection. Par exemple : `"Null"`, `"na"`, `""`, `" "`, `None` ou `"None"` sont convertis en valeurs manquantes reconnues par Pandas.

La méthode `.isnull()` est ensuite appliquée au `DataFrame` afin d'identifier les colonnes contenant des valeurs manquantes.

Si des valeurs nulles sont détectées, la suppression est effectuée à l'aide de la méthode `.dropna()`, qui élimine directement les lignes concernées du dataset.

2.2 Les valeurs aberrantes et différence d'écriture

Les différences d'écriture ont été identifiées lors de l'exploration des fichiers et des visualisations. Lorsqu'une incohérence est détectée, un traitement spécifique est ajouté au code.

Par exemple, dans la colonne `attack_cat`, les valeurs `Backdoor` et `Backdoors` désignent la même catégorie. Le traitement consiste à supprimer les espaces inutiles en début et fin de chaîne, puis à remplacer les variantes par une forme unique. Cette méthode est appliquée à chaque incohérence repérée afin d'assurer l'uniformité des catégories.

Concernant les valeurs aberrantes, des règles de validation précises ont été définies. Certaines variables ne peuvent prendre leurs valeurs que dans des intervalles spécifiques. Afin de centraliser ces règles et de rendre le nettoyage reproductible, un fichier `features.json` a été mis en place.

2.3 Nettoyage des données

Le processus de nettoyage est automatisé via une configuration centralisée. Ce fichier contient :

- les colonnes à conserver ;
- le type de traitement à appliquer ;
- les bornes ou règles associées.

Sa structure est la suivante :

```
{
  "nom_colonne_1": ["stat"],
  "nom_colonne_2": ["binary"],
  "nom_colonne_3": ["between", valeur_min, valeur_max],
  "nom_colonne_4": ["above", valeur_min],
  "nom_colonne_5": ["under_equal", valeur_max]
}
```

exemple.py

Les traitements possibles incluent notamment :

- **stat** : suppression des valeurs aberrantes via la méthode des interquartiles (IQR).
- **binary** : conservation des valeurs comprises entre 0 et 1.
- **between** : conservation des valeurs entre deux bornes spécifiques.
- **stat** : suppression des valeurs aberrantes via la méthode des interquartiles (IQR) ;
- **binary** : conservation des valeurs comprises entre 0 et 1 ;
- **between** : conservation des valeurs entre deux bornes ;
- **above / above_equal** : conservation des valeurs supérieures (strictement ou non) à un seuil ;
- **under / under_equal** : conservation des valeurs inférieures (strictement ou non) à un seuil.

Les colonnes non présentes dans ce fichier sont automatiquement supprimées. Cette organisation permet d'obtenir un nettoyage structuré, cohérent et facilement modifiable selon les besoins du projet.

Utilisation du code

Le script de nettoyage repose sur une arborescence fixe afin d'assurer un traitement automatisé et reproductible des datasets. Le répertoire principal doit contenir :

- un fichier `features.json`, définissant :
 - la liste des colonnes à conserver,
 - le type attendu pour chaque variable,
 - les règles de validation (bornes, traitement statistique, binaire, etc.) ;

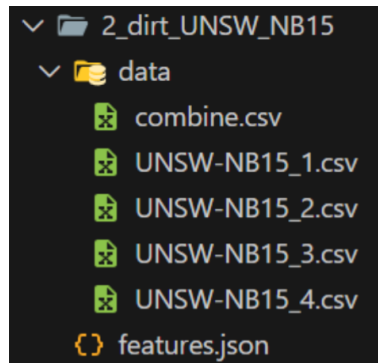


FIGURE 4 : exemple de répertoire

- un sous-répertoire data/ contenant un ou plusieurs fichiers de données au format CSV ou Parquet.

Le programme parcourt automatiquement le répertoire data/, détecte le type de fichier et applique la chaîne complète de nettoyage : suppression des colonnes non conformes, vérification des types, traitement des valeurs nulles, gestion des valeurs aberrantes et harmonisation de certaines variables textuelles.

À la fin du traitement, chaque fichier est réenregistré directement à son emplacement d'origine. Une fonction optionnelle permet également de fusionner tous les fichiers nettoyés en un unique dataset consolidé.

Le script modifie directement les fichiers fournis. Il est donc impératif :

- de conserver une copie des datasets originaux avant exécution ;
- de ne pas laisser les fichiers ouverts pendant le traitement ;
- de respecter strictement l'arborescence attendue.

Une attention particulière doit également être portée aux espaces parasites dans les noms de colonnes ou les valeurs textuelles, qui peuvent altérer la correspondance avec le fichier features.json.

3 Choix des métriques d'évaluation des modèles

Le choix des métriques d'évaluation est une étape importante dans le cadre de ce projet de détection d'intrusion car il ne suffit pas qu'un modèle affiche de bonnes performances globales il doit surtout être capable de détecter efficacement les attaques tout en limitant les fausses alertes. Dans notre travail, ces métriques ont été utilisées pour évaluer le TabTransformer ainsi que les autres modèles comparatifs, pour assurer une analyse cohérente et homogène. Les indicateurs retenus sont l'accuracy score, le F1 - score, la matrice de confusion et la courbe ROC associée à l'AUC. Ce choix est directement lié à la nature du problème, qui est une classification binaire (trafic normal vs trafic malveillant).

3.1 Accuracy score

L'accuracy mesure la proportion totale de prédictions correctes. Elle donne une première vision globale des performances et permet de comparer rapidement plusieurs modèles. Mais le soucis est que dans un contexte de cybersécurité, cette métrique peut être insuffisante si les classes sont déséquilibrées. Un modèle pourrait obtenir une accuracy élevée simplement en privilégiant la classe majoritaire. C'est pourquoi nous gardons cette métrique comme indicateur général mais sans en faire la métrique principale de décision.

3.2 F1-score

Le F1-score a été choisi pour apporter une vision plus pertinente dans le cadre de la détection d'intrusion. Il combine la précision et le rappel en une seule mesure. La précision mesure la fiabilité des alertes générées tandis que le rappel mesure la capacité à détecter effectivement les attaques. Le F1-score permet donc d'équilibrer ces deux aspects, et c'est essentiel dans un système de sécurité car un bon modèle doit détecter un maximum d'attaques sans générer trop de faux positifs.

3.3 Matrice de confusion

La matrice de confusion vient compléter ces scores en nous donnant des résultats détaillée des prédictions. Elle fait la différence entre les vrais positifs, vrais négatifs, faux positifs et faux négatifs. Dans un système de détection d'intrusion, les faux négatifs sont critiques car ils correspondent à des attaques non détectées, tandis que les faux positifs peuvent entraîner une surcharge d'alertes. L'analyse de cette matrice permet donc de comprendre précisément le comportement du modèle et le compromis qu'il réalise entre sensibilité et fiabilité.

3.4 Courbe ROC et AUC

Enfin, la courbe ROC et l'AUC permettent d'évaluer la capacité du modèle à distinguer les deux classes indépendamment d'un seuil fixe. La courbe ROC représente l'évolution du taux de vrais positifs en fonction du taux de faux positifs pour différents seuils. Et l'AUC transforme cette information en une seule valeur, plus elle est élevée, meilleure est la capacité de discrimination du modèle.

En combinant toutes ces métriques, nous obtenons une évaluation complète et adaptée aux contraintes de la détection d'intrusion. L'accuracy donne une vision globale, le F1-score met l'accent sur l'équilibre entre détection et fiabilité, la matrice de confusion permet une analyse plus fine des erreurs, et la courbe ROC avec l'AUC mesure la capacité de séparation des classes.

4 Modèle : Random Forest Classifier avec Équilibrage de Données

La principale difficulté de ce dataset réside dans son déséquilibre de classes extrême. Certaines catégories comme Generic comptent plusieurs dizaines de milliers d'échantillons, tandis que Worms n'en contient moins d'un dixième. Sans traitement adapté, tout modèle naïf aurait tendance à ignorer ces classes rares, précisément celles qui représentent les menaces les plus dangereuses.

4.1 Méthodologie

Pour traiter ce déséquilibre, nous avons construit une pipeline de traitement intégrée reposant sur la bibliothèque `imbalanced-learn`. Cette approche présente un avantage fondamental : elle enchaîne automatiquement les étapes de prétraitement, d'équilibrage et d'entraînement, tout en prévenant toute fuite de données (data leakage).

L'`ImbPipeline` est la colonne vertébrale du projet. Elle orchestre séquentiellement trois opérations critiques :

1. Transformation des colonnes (encodage des variables catégorielles, normalisation des variables numériques via `StandardScaler`).
2. Génération des données synthétiques par SMOTE, appliquée uniquement sur l'ensemble d'entraînement.
3. Entraînement du modèle sur ces données enrichies et équilibrées.

Ce cloisonnement est essentiel, si le SMOTE était appliqué manuellement sur l'ensemble du dataset avant la séparation train/test, le modèle aurait accès, de manière indirecte, à des informations issues du jeu de test. La pipeline garantit l'intégrité de l'évaluation en éliminant ce risque de triche.

Le SMOTE (Synthetic Minority Over-sampling Technique) est l'outil central pour corriger le déséquilibre des classes. Contrairement à une simple duplication de lignes existantes, SMOTE génère de nouveaux exemples artificiels en interpolant mathématiquement entre des points voisins dans l'espace des caractéristiques.

Concrètement, pour une classe aussi rare que Worms, cela signifie que le modèle ne se contente pas d'apprendre par cœur les 26 exemples originaux : il explore une région représentative de leur voisinage, acquérant ainsi une compréhension généralisable des signatures de cette attaque.

Le classificateur retenu est une forêt aléatoire de 350 arbres de décision, configurée avec plusieurs contraintes délibérées :

Paramètre	Valeur	Justification
n_estimators	350	Un nombre élevé d'arbres garantit la stabilité des prédictions par effet de vote majoritaire. La variance du modèle est ainsi réduite.
max_depth	15	En limitant la profondeur de chaque arbre, on évite le sur-apprentissage (overfitting). Sans cette contrainte, chaque arbre pourrait mémoriser parfaitement les données d'entraînement y compris leurs bruits au détriment de sa capacité à généraliser.
min_samples_leaf	5	Chaque feuille doit représenter au minimum 5 exemples, ce qui lisse les décisions aux extrémités de l'arbre.
class_weight	balanced_subsample	À chaque tirage d'arbre, les poids sont dynamiquement recalculés pour accorder une pénalité plus forte aux erreurs commises sur les classes minoritaires. Une mauvaise classification d'un Worm est ainsi bien plus coûteuse qu'une erreur sur un Generic.

4.2 Résultats et Analyse

Le modèle atteint une précision globale (accuracy) de 86% sur l'ensemble de validation. Ce chiffre, bien qu'encourageant, doit être interprété avec nuance au regard du déséquilibre des classes.

Classe	Précision	Rappel	F1-Score	Interprétation
Generic	1.00	0.97	0.99	Performance quasi parfaite sur le trafic sain.
Fuzzers	0.88	0.83	0.85	Excellente détection, modèle très fiable sur cette classe.
Reconnaissance	0.81	0.81	0.81	Résultat solide et équilibré.
Shellcode	0.30	0.86	0.44	Rappel élevé : l'attaque est bien détectée, au prix de quelques fausses alertes.
Worms	0.05	0.77	0.10	Succès du SMOTE : 77% des vers sont détectés, contre 23% sans équilibrage.

Ces résultats illustrent un arbitrage fondamental en cybersécurité : le compromis entre précision et rappel. Dans ce contexte applicatif, une fausse alerte (précision faible) est bien moins coûteuse qu'une attaque non détectée (rappel faible). C'est pourquoi les choix de conception ont délibérément privilégié un rappel élevé sur les classes critiques, quitte à accepter une précision plus modeste pour Shellcode et Worms.

L'impact du SMOTE est particulièrement visible sur la classe Worms : sans équilibrage, le modèle ne détectait que 23% de ces attaques, après optimisation, ce taux monte à 77%, ce qui représente un gain opérationnel considérable.

Le modèle développé constitue un système de détection d'intrusions performant et opérationnel. Il excelle sur les classes bien représentées (Generic, Fuzzers, Reconnaissance) et parvient à détecter la grande majorité des attaques rares grâce à la combinaison du SMOTE et du balanced_subsample.

5 **Modèle : XGBoost**

5.1 Préparation des Données

XGBoost, contrairement au Random Forest, impose que la variable cible soit exprimée sous forme d'entiers consécutifs. L'utilisation d'un LabelEncoder est donc une étape obligatoire, elle transforme les étiquettes textuelles des catégories d'attaques (Generic, Worms, Shellcode...) en indices numériques (0, 1, 2...) exploitables par l'algorithme.

Lors de la séparation des données en ensembles d'entraînement, de test et de validation, le paramètre `stratify=y_encoded` a été maintenu. Cette précaution garantit que les classes ultra-minoritaires au premier rang desquelles Worms sont représentées dans chacun des trois ensembles, évitant ainsi qu'une classe entière soit absente du jeu d'entraînement par effet du hasard.

Dans un premier temps, la fonction `compute_class_weight` calcule pour chaque classe un poids mathématiquement inversement proportionnel à sa fréquence d'apparition. Les classes rares se voient ainsi attribuer un coefficient bien plus élevé que les classes dominantes. Ces poids sont ensuite injectés directement dans la méthode `.fit()` via le paramètre `sample_weight`, ce qui amène XGBoost à pénaliser beaucoup plus lourdement toute erreur commise sur un échantillon minoritaire. Concrètement, une mauvaise classification d'un Worm devient aussi coûteuse pour le modèle que des centaines d'erreurs sur du trafic Generic, le forçant à développer une attention particulière envers ces cas difficiles.

5.2 Architecture du Modèle et Choix des Hyperparamètres

La configuration de XGBoost résulte d'un ensemble de choix délibérés, chacun répondant à une contrainte spécifique du dataset.

5.3 Suivi de la Convergence

Le modèle a été configuré avec un `eval_set` pointant vers l'ensemble de validation, permettant de surveiller la progression de l'apprentissage à chaque itération. La métrique suivie est le `mlogloss` (Multi-class Logarithmic Loss) : cette fonction de perte mesure non seulement si le modèle se trompe, mais également à quel point il est certain de ses prédictions. Plus elle est basse, plus le modèle est confiant dans ses bonnes classifications.

Les résultats montrent une décroissance régulière du `mlogloss` jusqu'à 0.35, sans plateau ni remontée preuve d'une convergence saine et de l'absence de sur-apprentissage majeur au cours de l'entraînement.

Paramètre	Valeur	Justification
n_estimators	500	Un nombre élevé d'arbres laisse au modèle le temps de corriger ses erreurs sur les classes complexes, après avoir maîtrisé les plus simples.
learning_rate	0.05	Un taux d'apprentissage réduit (initialement 0.1) ralentit la progression du modèle pour lui permettre de converger vers un minimum d'erreur plus précis, sans sauter par-dessus.
max_depth	8	Une profondeur plus grande que dans le Random Forest (15 niveaux avec contrainte SMOTE) est nécessaire ici pour capturer les interactions fines entre variables réseau en l'absence de sur-échantillonnage.
subsample	0.8	Chaque arbre n'est entraîné que sur 80% des données, tirés aléatoirement. Cette diversité structurelle empêche le modèle de mémoriser le bruit du dataset.
gamma	1	Ce paramètre de régularisation interdit la création d'une nouvelle branche si le gain en précision associé est trop faible, limitant efficacement le sur-apprentissage.
tree_method	'hist'	L'algorithme par histogramme accélère drastiquement l'entraînement sur des datasets de plus de 300 000 lignes, sans perte de qualité.

5.4 Résultats et Analyse

✓ Points forts

La pondération des échantillons combinée aux 500 estimateurs a produit un gain significatif sur les classes les plus difficiles. Le modèle, contraint de traiter chaque erreur sur Worms ou Shellcode comme une faute critique, a développé une spécialisation sur ces signatures rares qui se traduit par une nette progression du rappel sur ces catégories par rapport à la version initiale non pondérée.

La fiabilité sur les classes majeures Generic, Fuzzers, Reconnaissance reste totale, ce qui confirme que l'optimisation pour les classes minoritaires n'a pas dégradé les performances globales du système.

⚠ Points faibles

La classe Analysis continue de présenter un score faible, quelle que soit la configuration testée. Cette limite n'est pas imputable aux choix de modélisation, les caractéristiques réseau de ce type d'attaque dans le dataset UNSW-NB15 sont statistiquement trop proches de celles du trafic normal pour permettre une discrimination fiable. Il s'agit d'une contrainte inhérente aux données elles-mêmes, qui ne pourrait être levée que par l'ajout de nouvelles variables discriminantes ou une source de données complémentaire.

Le résultat est un modèle robuste, atteignant le meilleur compromis Précision/Rappel obtenu sur ce projet capable de détecter les attaques rares tout en maintenant une fiabilité totale sur le trafic sain, et donc pleinement adapté à un déploiement dans un contexte opérationnel de cybersécurité.

6 **Modèle : TabNet**

6.1 Préparation des Données

Les réseaux de neurones sont fondamentalement plus sensibles à la qualité et à la structure des données que les modèles à base d'arbres. Plusieurs précautions spécifiques ont donc été appliquées en amont de l'entraînement.

Une correction préalable a consisté à travailler systématiquement sur des copies explicites des DataFrames (`.copy()`), afin d'éviter le `SettingWithCopyWarning` de Pandas. Ce point, souvent négligé, est essentiel : sans cette précaution, les transformations appliquées (normalisation, encodage) risquent de corrompre silencieusement les données originales par effet de référence partagée, introduisant des incohérences difficiles à diagnostiquer.

6.1.1 Normalisation des variables numériques

TabNet repose sur des calculs de gradient pour ajuster ses paramètres internes. Or, si les variables numériques présentent des échelles très hétérogènes — comme c'est le cas entre `sload` (charge en bits par seconde) et `sbytes` (volume en octets) — les variables à grandes valeurs dominent mécaniquement le signal d'apprentissage et empêchent le modèle de converger. L'application d'un `StandardScaler` sur toutes les colonnes numériques ramène chaque variable à une moyenne nulle et un écart-type unitaire, rétablissant un traitement équitable entre toutes les caractéristiques.

6.1.2 Embeddings pour les variables catégorielles

Les colonnes catégorielles (protocole, état, service) ont été identifiées dynamiquement via les paramètres `cat_idx`s et `cat_dim`s. TabNet ne se contente pas d'un simple encodage numérique : il apprend des embeddings, c'est-à-dire des représentations vectorielles denses qui capturent des relations sémantiques entre les modalités. Le modèle peut ainsi percevoir que TCP et UDP partagent certaines propriétés structurelles, une nuance inaccessible à un encodage one-hot classique.

6.2 Architecture et Mécanisme d'Attention

6.2.1 Sélection parcimonieuse des caractéristiques avec Sparsemax

Le choix de `mask_type='sparsemax'` est une décision architecturale centrale. Contrairement à une fonction Softmax classique qui distribue toujours un poids non nul à toutes les caractéristiques, Sparsemax force une partie des poids à zéro. À chaque étape de décision, le modèle sélectionne activement un sous-ensemble restreint de variables jugées pertinentes pour l'échantillon traité, ignorant explicitement les autres. Ce mécanisme confère à TabNet une

forme d'attention sélective et interprétable, et améliore la généralisation en réduisant le bruit issu des variables non informatives.

6.2.2 Stabilisation de l'apprentissage par Virtual Batch Normalization

L'entraînement utilise un `batch_size` de 4 096 échantillons, au sein duquel une normalisation est appliquée sur des sous-lots (virtual batches) de 256 échantillons. Ce concept, issu de la Ghost Batch Normalization, améliore la régularisation implicite du modèle et stabilise les mises à jour des gradients, particulièrement bénéfique sur un dataset aussi hétérogène que UNSW-NB15.

6.3 Stratégie d'Optimisation

6.3.1 Pondération automatique des classes

Le paramètre `weights=1` passé à la méthode `.fit()` active l'équilibrage automatique des classes par TabNet. Le modèle calcule dynamiquement l'importance relative de chaque classe et ajuste sa fonction de perte en conséquence un mécanisme analogue au `balanced_subsample` du Random Forest ou au `sample_weight` de XGBoost, mais intégré nativement à l'architecture. C'est ce rééquilibrage qui explique en grande partie le rappel exceptionnel de 0.88 obtenu sur la classe Worms.

6.3.2 Décroissance du taux d'apprentissage

Un scheduler de type StepLR réduit progressivement le taux d'apprentissage tous les 10 paliers (`gamma=0.9`). Cette stratégie suit une logique en deux temps : le modèle apprend d'abord les tendances générales avec un taux élevé, puis affine sa compréhension des zones de décision complexes à mesure que le taux diminue. Ce réglage progressif est particulièrement précieux pour les classes dont les frontières de décision sont floues, comme Analysis ou Backdoor.

6.4 Analyse des Résultats

TabNet présente un profil de performance distinct de celui des deux modèles précédents, avec des points forts propres à son architecture.

Le résultat le plus notable est l'amélioration sur la classe Analysis, là où les méthodes ensemblistes (Random Forest, XGBoost) échouaient en raison de la proximité statistique de cette attaque avec le trafic normal, le mécanisme d'attention de TabNet parvient à isoler des combinaisons de caractéristiques plus subtiles, inaccessibles à une approche par arbres de décision.

TabNet apporte une dimension qualitativement différente au projet, là où Random Forest et XGBoost s'appuient sur l'agrégation de règles de décision, TabNet opère un filtrage intelligent

Dimension	Résultat	Interprétation
Rappel (classes critiques)	Exceptionnel sur Shellcode et Worms	Meilleure couverture sécuritaire parmi les trois modèles.
Précision sur Analysis	0.23	Supérieur à XGBoost et Random Forest : l'attention par Sparsemax capte mieux la complexité de cette classe.
Faux positifs	Élevés sur les classes ultra-minoritaires	Conséquence directe de la sensibilité maximale du modèle : un compromis assumé.

des caractéristiques, adapté échantillon par échantillon. Cette flexibilité se traduit par une sensibilité accrue aux signaux faibles, les attaques rares et atypiques qui en fait le candidat le plus adapté à un système de détection d'intrusion axé sur la couverture sécuritaire maximale.

7 Modèle : TabTransformer

7.1 Qu'est-ce qu'un TabTransformer ?

Le TabTransformer est un modèle de deep learning conçu spécifiquement pour les données tabulaires. Contrairement aux modèles ensemblistes à base d'arbres comme le Random Forest ou XGBoost, qui traitent chaque feature de manière indépendante, le TabTransformer utilise un Transformer pour capturer les interactions entre features catégorielles.

Le Transformer est une architecture de réseau de neurones introduite par Vaswani et al. en 2017 dans l'article « Attention Is All You Need ». Son innovation principale est le mécanisme de self-attention : au lieu de traiter les données séquentiellement (comme les réseaux récurrents RNN ou LSTM), il examine tous les éléments en parallèle et calcule un score d'importance entre chaque paire, répondant à la question « quels éléments dois-je regarder pour mieux comprendre celui-ci ? ». Initialement développé pour le traitement du langage naturel (traduction, génération de texte), le Transformer a depuis été adapté à de nombreux domaines dont la vision par ordinateur et, ici, les données tabulaires.

L'idée du TabTransformer est d'appliquer ce mécanisme aux colonnes catégorielles d'un tableau. Dans notre cas, un flux réseau est décrit par un protocole (TCP, UDP...), un service (HTTP, FTP...) et un état de connexion (FIN, CON...). Plutôt que de traiter ces trois informations séparément, le TabTransformer permet au modèle d'apprendre que certaines combinaisons sont caractéristiques de certains types d'attaques (par exemple, un protocole TCP avec un service HTTP dans un état CON).

L'architecture se décompose en trois blocs principaux :

- **Column Embedding** : transformation des valeurs catégorielles en vecteurs numériques de 32 dimensions.
- **Transformer Encoder** : 6 couches de self-attention multi-têtes qui apprennent les relations

entre ces vecteurs.

- **MLP de classification** : réseau de neurones classique qui combine la sortie du Transformer avec les features numériques pour produire la prédiction finale.

7.2 Préparation des Données

Le TabTransformer, comme tout réseau de neurones, nécessite un prétraitement rigoureux des données d'entrée. La préparation s'articule autour de trois étapes.

Encodage des variables catégorielles : Chaque colonne catégorielle est encodée via un LabelEncoder qui associe un entier unique à chaque modalité. Le modèle dispose de 3 features catégorielles : proto (129 catégories), service (13 catégories) et state (6 catégories). Contrairement à un encodage one-hot, le TabTransformer utilise ensuite des embeddings appris qui capturent les relations sémantiques entre modalités (par exemple, TCP et UDP partagent certaines propriétés structurelles).

Normalisation des variables numériques : Les 18 features numériques sont standardisées par un StandardScaler (moyenne nulle, écart-type unitaire) pour éviter que les variables à grande échelle (sload en bits/s) ne dominent celles à petite échelle (ct_state_ttl). Un LayerNorm supplémentaire est appliqué à l'intérieur du modèle.

Séparation des données : Le dataset de 321 283 lignes est découpé en trois ensembles stratifiés (même distribution des classes dans chaque split) : Train 70% (224 889), Validation 15% (48 201) et Test 15% (48 193). La stratification garantit la présence des classes rares (Worms : 174 échantillons au total) dans chaque split.

7.3 Architecture et Mécanisme d'Attention

Column Embedding : Chaque feature catégorielle est transformée en vecteur numérique par la concaténation d'un *Value Embedding* (vecteur appris de 28 dimensions, unique par valeur : proto=tcp $\neq$ proto=udp) et d'un *Column Identifier* (vecteur fixe de 4 dimensions identifiant la colonne d'origine). Cela produit un vecteur de 32 dimensions par feature. Avec 3 catégorielles, on obtient une séquence de 3 tokens $\times$ 32 dimensions en entrée du Transformer.

Transformer Encoder (Self-Attention Multi-Têtes) : Chaque token est projeté en trois vecteurs : *Query* (« ce que je cherche »), *Key* (« ce que je suis ») et *Value* (« l'information que je porte »). Le score d'attention est calculé par le produit scalaire $QK^T / \sqrt{d}$, normalisé par softmax. Ce calcul est répété par 8 têtes d'attention en parallèle (chacune sur 4 dimensions), permettant d'apprendre différents types de relations entre features. 6 couches sont empilées, chacune complétée par un réseau feedforward (32 $\rightarrow$ 128 $\rightarrow$ 32, activation GELU), des connexions résiduelles et un LayerNorm. La sortie est aplatie en un vecteur de $3 \times 32 = 96$ dimensions.

MLP de classification : Un MLP (Multi-Layer Perceptron) est un empilement de couches « fully connected » où chaque neurone reçoit toutes les sorties de la couche précédente, les multiplie

par des poids appris, et applique une fonction d'activation non-linéaire (ReLU : $\max(0, x)$). Sans cette non-linéarité, un empilement de couches équivaldrait à une seule couche linéaire. Le MLP prend la concaténation de la sortie Transformer (96 dim) et des 18 features numériques normalisées, soit 114 dimensions, puis passe par $114 \rightarrow 256 \rightarrow 128 \rightarrow 9$ neurones (BatchNorm + ReLU + Dropout 0.3 entre chaque couche). La sortie produit 9 scores de confiance, un par classe d'attaque.

7.3.1 Hyperparamètres

Paramètre	Valeur	Justification
embed_dim / heads / layers	32 / 8 / 6	Conforme au paper. 8 têtes $\times$ 4 dim/tête, 6 couches de profondeur.
learning_rate / optimizer	1e-4 / AdamW	Weight decay 1e-4. Scheduler : $\div 2$ après 5 epochs sans amélioration.
batch_size / epochs	512 / 100	Early stopping patience=15 sur le Macro-F1 de validation.
dropout	0.1 / 0.3	0.1 dans le Transformer, 0.3 dans le MLP (plus agressive).
loss	CrossEntropy	Pondérée par $1/\sqrt{\text{fréquence}}$. Une erreur sur Worms coûte plus qu'une erreur sur Generic.
features	18 NUM + 3 CAT	Sélection par scoring ANOVA + MI + RF + Cramér's V. Total : 240 565 paramètres.

7.4 Résultats et Analyse

Classe	Précision	Rappel	F1-score	Support	Interprétation
Generic	1.00	0.97	0.99	32 322	Performance quasi parfaite sur la classe majoritaire.
Fuzzers	0.83	0.81	0.82	3 637	Excellente détection, modèle fiable.
Reconnaissance	0.78	0.80	0.79	2 098	Résultat solide et équilibré.
Exploits	0.79	0.53	0.63	6 679	Bonne précision mais confusions avec DoS.
Shellcode	0.37	0.70	0.48	227	Rappel élevé malgré la rareté (0.5% du dataset).
DoS	0.33	0.73	0.46	2 453	Rappel fort mais beaucoup de faux positifs.
Analysis	0.36	0.27	0.31	402	Signatures trop proches du trafic normal.
Worms	0.13	0.54	0.22	26	54% détectés malgré seulement 26 samples en test.
Backdoor	0.12	0.07	0.09	349	Classe la plus difficile à distinguer.

Accuracy	Macro-F1	Weighted-F1	ROC-AUC	PR-AP
0.87	0.53	0.87	0.971	0.539

✓ Points forts

Le ROC-AUC macro de 0.971 est le résultat le plus significatif : il démontre que le modèle apprend des représentations discriminantes pour toutes les classes, y compris les plus rares (Worms : AUC = 0.990, Shellcode : AUC = 0.993). Ce score, calculé sur toute la plage de seuils possibles, montre que les probabilités produites par le modèle sont bien calibrées. Les classes bien représentées (Generic, Fuzzers, Reconnaissance) obtiennent des performances comparables à celles du Random Forest et de XGBoost, avec l'interprétabilité en plus.

⚠ Points faibles

La classe Backdoor reste la plus problématique (F1 = 0.09), une difficulté partagée avec les autres modèles. Les signatures réseau de cette catégorie dans UNSW-NB15 sont statistiquement trop proches du trafic normal pour permettre une discrimination fiable avec les features disponibles. L'écart entre accuracy (0.87) et Macro-F1 (0.53) reflète le déséquilibre extrême du dataset : l'accuracy est tirée vers le haut par Generic (67% des données), tandis que le Macro-F1 pénalise les performances sur les classes représentant moins de 1%.

7.5 Interprétabilité : Analyse des Attention Maps

L'un des avantages distinctifs du TabTransformer par rapport aux modèles ensemblistes est la possibilité d'extraire les poids d'attention pour comprendre le raisonnement du modèle. En récupérant ces poids via des hooks sur chaque couche du Transformer, moyennés sur les 8 têtes et 200 échantillons par classe, on obtient une carte d'attention montrant quelles features catégorielles le modèle relie entre elles pour chaque type d'attaque.

Classe	proto → service	proto → state	service → state
Analysis	0.354	0.347	0.339
Backdoor	0.356	0.344	0.344
Exploits	0.341	0.355	0.340
Fuzzers	0.364	0.342	0.342
Generic	0.271	0.435	0.421
Reconnaissance	0.362	0.343	0.340
Worms	0.354	0.347	0.328

Le résultat le plus remarquable concerne la classe Generic : le modèle a appris à fortement

réduire l'attention sur le protocole ($\text{proto} \rightarrow \text{service} = 0.271$, bien en dessous de la distribution uniforme à 0.333) au profit de l'état de connexion ($\text{proto} \rightarrow \text{state} = 0.435$, $\text{service} \rightarrow \text{state} = 0.421$). Cela signifie que pour détecter les attaques Generic, le *state* (FIN, CON, INT) est le signal le plus discriminant, indépendamment du protocole utilisé. Ce résultat est cohérent : les attaques Generic représentent 67% des données et utilisent tous les protocoles, rendant *proto* non informatif pour cette classe.

Pour les autres classes, l'attention reste quasi-uniforme (≈ 0.34), ce qui est attendu : le Tab-Transformer est conçu pour des datasets avec 10 à 20+ features catégorielles. Avec seulement 3 tokens, le mécanisme d'attention n'a pas assez de matière pour apprendre des patterns complexes. L'essentiel du pouvoir prédictif provient alors du MLP appliqué aux 18 features numériques. Cette analyse démontre néanmoins l'interprétabilité du TabTransformer : là où les modèles ensemblistes fonctionnent en « boîte noire », le mécanisme d'attention offre une fenêtre directe sur la stratégie de décision du modèle.

7.6 Tests de Robustesse

Un modèle performant sur un benchmark n'est pas nécessairement fiable en production. Deux tests évaluent la stabilité du TabTransformer face à des conditions dégradées.

7.6.1 Sous-sampling (réduction du volume d'entraînement)

Le modèle est ré-entraîné avec 25%, 50%, 75% et 100% des données (50 epochs, même architecture). Le test set reste identique.

Fraction	Lignes train	Macro-F1	Accuracy
25%	56 222	0.4509	0.8553
50%	112 444	0.4682	0.8602
75%	168 667	0.4861	0.8634
100%	224 889	0.5004	0.8605

La dégradation est progressive : le Macro-F1 passe de 0.50 (100%) à 0.45 (25%), soit une perte de 10%. L'accuracy reste stable (≈ 0.86), montrant que la classe majoritaire (Generic) est toujours bien classifiée même avec peu de données. La perte se concentre sur les classes rares.

7.6.2 Sensibilité au bruit

Du bruit gaussien ($\sigma = 0.05$ à 0.30) est ajouté aux features numériques standardisées du test set. Le modèle déjà entraîné est évalué sans ré-entraînement, simulant des données corrompues en production (capteurs défectueux, données incomplètes).

Le modèle est sensible au bruit sur les features numériques : une perturbation de 10% ($\sigma =$

Bruit (σ)	Macro-F1	Accuracy	Dégradation F1
0.00 (baseline)	0.5311	0.8668	—
$\sigma = 0.05$	0.4698	0.8505	−11.5%
$\sigma = 0.10$	0.4034	0.8192	−24.0%
$\sigma = 0.20$	0.3340	0.7482	−37.1%
$\sigma = 0.30$	0.2946	0.6906	−44.5%

0.10) entraîne une chute de 24% du Macro-F1. Ce résultat est cohérent avec l'architecture : les 18 features numériques traversent le MLP sans passer par le Transformer, et n'ont donc pas de mécanisme de lissage ou de contextualisation pour absorber le bruit. À $\sigma = 0.30$, l'accuracy tombe à 0.69, montrant que même la détection de Generic est affectée. Ces résultats soulignent l'importance de la qualité des données en entrée pour un déploiement en production.

8 Conclusion

Ce projet a exploré quatre approches complémentaires pour la détection d'intrusions sur le dataset UNSW-NB15 : Random Forest, XGBoost, TabNet et TabTransformer. Chaque modèle apporte une réponse différente au défi central du déséquilibre extrême des classes et de la diversité des signatures d'attaques.

Modèle	Accuracy	Force principale	Approche
Random Forest	86%	Stabilité, simplicité	Agrégation de 350 arbres + SMOTE pour les classes rares.
XGBoost	Meilleur compromis P/R	Précision/Rappel	Boosting séquentiel + pondération par fréquence inverse.
TabNet	—	Rappel classes rares	Attention Sparsemax + sélection dynamique des features.
TabTransformer	87%	Interprétabilité	Self-attention sur features catégorielles + MLP sur numériques.

Les modèles ensemblistes (Random Forest, XGBoost) excellent par leur fiabilité sur les classes bien représentées et leur rapidité d'entraînement. Les modèles à base d'attention (TabNet, TabTransformer) apportent une dimension supplémentaire : la capacité d'expliquer leurs décisions. Le TabTransformer, via ses attention maps, révèle par exemple que l'état de connexion (state) est le signal le plus discriminant pour la classe Generic, une information directement exploitable par un analyste SOC.

Un constat traverse les quatre modèles : les classes Backdoor et Analysis restent difficiles à classifier, quelle que soit l'architecture utilisée. Cette limite est inhérente au dataset UNSW-NB15 lui-même, dont les signatures réseau pour ces catégories sont statistiquement trop

proches du trafic normal. Aucune optimisation d'hyperparamètres ne peut compenser un manque de signal dans les données.

Les tests de robustesse conduits sur le TabTransformer confirment une bonne stabilité face à la réduction de volume (−10% de Macro-F1 à 25% des données) mais une sensibilité au bruit (−24% à $\sigma=0.10$), soulignant l'importance de la qualité des données en entrée pour tout déploiement opérationnel.

Plusieurs pistes d'amélioration sont envisagées pour la suite du projet :

- **FT-Transformer** : transformer toutes les features en tokens pour exploiter pleinement le mécanisme d'attention.
- **Discrétisation de features numériques** (quantile binning) pour enrichir la séquence de tokens au-delà des 3 catégorielles actuelles.
- **Sur-échantillonnage ciblé** (SMOTE) sur Worms, Backdoor et Analysis, combiné aux architectures deep learning.
- **Validation multi-dataset** (CIC-IDS2017, CSE-CIC-IDS2018) pour confirmer la transférabilité des modèles à d'autres environnements réseau.
- **Approche ensembliste hybride** : combiner les prédictions des quatre modèles par vote pondéré pour capitaliser sur les forces de chacun.